1 Combinators

1.1 Terms and Reduction

A type checker does not care what our function computes, only what it *takes* and what it *returns*;
It ignores expression values, library function names, keywords, numbers - all is stripped away and very little is left.

Definition 1.1 (terms, definitions, combinations)**.** Fix two *constants*, **K** and **S**, and a stock of *variables* $x, y, z, f, g, \ldots$

1. Each constant, **K** and **S**, is a term
2. Each variable x is a term.
3. If M and N are terms, then $M\,N$ is a term: the *application* of M to N
4. Nothing else is a term.

A *definition* is an equation:

$$f\,x_1\,\ldots\,x_n\;:=\;M,$$

where f is a new name, $x_1, \ldots, x_n$ are distinct variables, and M is a term built from constants, names defined earlier, and the variables $x_1, \ldots, x_n$ alone. Once defined, the name f may be used as a term.

The symbol := means "is defined to be", and is not equal to the = in standard arithmetic. In this course, it means "I am naming this", and claims nothing else. The last clause of the definition of a term is what "inductively" means, or in one line, with M and N standing for terms already built:

$$M, N ::= K \mid S \mid x \mid M\,N$$

Applications group to the left: $f\,x\,y$ is really $(f\,x)\,y$, a function of two arguments taking them one at a time;
Brackets are written only where the grouping is **to the right**, as in $f\,(g\,x)$;
In Haskell, the term language is `data Term = K | S | Var String | App Term Term`, one constructor per clause.

A term is *closed* if no variable occurs in it, and a closed term is called a *combinator*: in a closed term, there is nothing left to supply;
So, **S K K** and **K (S K)** are both combinators;
A term with variables in it, such as $f\,(g\,x)$ or $\mathbf{S}\,x\,\mathbf{K}$, is a *fragment*, it says what to do *once* its variables are given, and until they are given there is nothing to run.

Example 1.2 (definitions we have already written)**.** Three definitions in the sense of Definition 1.1:

$$twice\, f\, x := f(f\, x), \quad comp\, f\, g\, x := f(g\, x), \quad flip\, f\, x\, y := f\, y\, x$$

In Haskell, they are:
`twice f x = f (f x)`, `comp f g x = f (g x)`, and `flip f x y = f y x`;
in Python, three `def`s; in Javascript, `const twice = f => x => f(f(x))` and its two equivalents.

What a definition may *not* do is reach outside itself. The RHS may use the two constants, parameters, and names defined earlier and nothing else.
Therefore, $flip\, f\, x\, y := f\, y\, x$ is a definition, but $bad\, f\, x := f\, (g\, x)$ is not, since g is neither a parameter nor a name defined earlier; and neither is is $bad\, f\, f := f$, whose parameters are not distinct.

A definition is a rule for rewriting: wherever f stands next to n arguments, the LHS may be replaced with the right, and arguments written into parameters.

Definition 1.3 (running)**.** A step rewrites a term in one of three ways:

$$\begin{aligned} \mathbf{K}\, M\, N &\rightsquigarrow M, \\ \mathbf{S}\, M\, N\, P &\rightsquigarrow M\, P\, (N\, P), \\ f\, M_1\, \ldots\, M_n &\rightsquigarrow M \text{ with } M_i \text{ in place of each } x_i \end{aligned}$$

the third whenever $f\, x_1 \ldots\, x_n := M$ is a definition. The part rewritten may sit anywhere inside a larger term.
A term which no step can be taken is in *normal form*.
To run a term is to take steps until normal form $M \rightsquigarrow^* N : N$ is reachable from M, zero steps included.

K is the function that keeps its first argument and drops the second:

$$\mathbf{K}\, M\, N \rightsquigarrow M$$

S is the function that hands its third argument to both others and applies the results:

$$\mathbf{S}\, M\, N\, P \rightsquigarrow M\, P\, (N\, P)$$

We can see why two constants of this kind is enough. All a variable does is get used once, many times, or not at all. The two terms **K** and **S** are Schönfinkel's basic combinators, terms and rewrite rules make a combinatory algebra.

Theorem 1.4 (two constants suffice; on loan)**.** *Every definition by equations can be replaced by a combinator in* ***K*** *or* ***S*** *alone that runs the same way on every argument; and the functions on numerals that such combinators compute are exactly the functions a Turing machine computes.*

This says that combinators **are** programs. We prove in the next few examples.

Example 1.5 (the identity, and composition)**.** Define the function that returns its arguments unchanged is **not** one of the two constants, but it is in the language. Define

$$I := \mathbf{S}\ \mathbf{K}\ \mathbf{K},$$

a definition with no parameters, and run it on an arbitrary argument x

$$\begin{aligned} \mathbf{S}\,\mathbf{K}\,\mathbf{K}\,x &\rightsquigarrow \mathbf{K}\,x\,(\mathbf{K}\,x) && \mathbf{S} \text{ step} \\ &\rightsquigarrow x && \mathbf{K} \text{ step} \end{aligned}$$

Whatever x is, $\mathbf{I}$ runs to x.
The *comp* of Example 1.2 can be written from the constants alone too.
Let $\mathbf{B} := \mathbf{S}\ (\mathbf{K}\ \mathbf{S})\ \mathbf{K}$ and run it on three arguments:

$$\begin{aligned} \mathbf{S}\,(\mathbf{K}\,\mathbf{S})\,\mathbf{K}\,f\,g\,x &\rightsquigarrow \mathbf{K}\,\mathbf{S}\,f\,(\mathbf{K}\,f)\,g\,x && \mathbf{S} \text{ step} \\ &\rightsquigarrow \mathbf{S}\,(\mathbf{K}\,f)\,g\,x && \mathbf{K} \text{ step} \\ &\rightsquigarrow \mathbf{K}\,f\,x\,(g\,x) && \mathbf{S} \text{ step} \\ &\rightsquigarrow f\,(g\,x) && \mathbf{K} \text{ step} \end{aligned}$$

Four steps, and we get the *comp* we wrote in Haskell, without a single parameter.

To watch a definition run we want something visible, so we allow the integers $0, 1, 2, \ldots$ and the operator $+$ as ready-made constants, with the obvious step $m + n \rightsquigarrow$ the sum. They are not terms of Definition 1.1, and the debt is paid at the end of this section.

$$\begin{aligned} \mathit{twice}\ \mathit{inc}\ 0 &\rightsquigarrow \mathit{inc}\,(\mathit{inc}\,0) && \text{definition of } \mathit{twice} \\ &\rightsquigarrow \mathit{inc}\,(0+1) && \text{definition of } \mathit{inc}\text{, inner} \\ &\rightsquigarrow \mathit{inc}\,1 && 0+1 \\ &\rightsquigarrow 1+1 && \text{definition of } \mathit{inc} \\ &\rightsquigarrow 2 && 1+1 \end{aligned}$$

That is all a function call does: replace the LHS by the RHS.

Example 1.6 (booleans)**.** A boolean is a choice between two things, so let *true* be the function that keeps the first of two args, and *false* the one that keeps the second.

$$\mathit{true} := \mathbf{K}, \qquad \mathit{false} := \mathbf{K}\,\mathbf{I}$$

With that choice, `if b then M else N` is simply $b\ M\ N$: the boolean applied to the two branches. Run both cases:

$$\begin{aligned} \mathit{true}\ M\ N = \mathbf{K}MN &\rightsquigarrow M && \text{keep the first} \\ \mathit{false}\ M\ N = \mathbf{K}\,\mathbf{I}\,M\,N &\rightsquigarrow \mathbf{I}\,N && \mathbf{K} \text{ keeps } \mathbf{I}\text{, drops } M \\ &\rightsquigarrow^{*} N && \text{Example 1.5} \end{aligned}$$

There is no `if` in the language, none is needed: the test chooses the branch by what it does with its two arguments. The boolean *is* the conditional.

Example 1.7 (numerals, and addition)**.** A number n is "do it n times": a function that takes an f and an x and applies f to x that many times. One definition for each n,

$$\bar{0}\, f\, x := x, \quad \bar{1}\, f\, x := f\, x, \quad \bar{2}\, f\, x := f(f\, x), \quad \bar{3}\, f\, x := f(f(f\, x)), \quad \ldots$$

To add one to a number, apply f once more; to add two numbers, apply f the first number of times to the result of applying it the second number of times:

$$\mathit{succ}\, n\, f\, x := f(n\, f\, x), \qquad \mathit{add}\, m\, n\, f\, x := m\, f\, (n\, f\, x).$$

Here is *add* at work, run on *inc* and 0 so that a number comes out, one step per line, with the definition unfolded shown on the right: Run $\mathit{add}\, \bar{2}\, \bar{1}$ on *inc* and 0.

$$\begin{aligned}
\mathit{add}\, \bar{2}\, \bar{1}\, \mathit{inc}\, 0 &\rightsquigarrow \bar{2}\, \mathit{inc}\, (\bar{1}\, \mathit{inc}\, 0) && \mathit{add}\\
&\rightsquigarrow \mathit{inc}\, (\mathit{inc}\, (\bar{1}\, \mathit{inc}\, 0)) && \bar{2}\\
&\rightsquigarrow \mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, 0)) && \bar{1}\\
&\rightsquigarrow \mathit{inc}\, (\mathit{inc}\, (0+1)) && \mathit{inc}\text{, innermost}\\
&\rightsquigarrow \mathit{inc}\, (\mathit{inc}\, 1) \rightsquigarrow \mathit{inc}\, (1+1) && 0+1;\ \mathit{inc}\\
&\rightsquigarrow \mathit{inc}\, 2 \rightsquigarrow 2+1 \rightsquigarrow 3 && \ldots
\end{aligned}$$

The answer is 3, exactly what $\bar{3}$ computes. An exercise:

$$\text{let } \mathit{mult}\, m\, n\, f := m(n\, f).$$

Run $\mathit{mult}\, \bar{2}\, \bar{3}\, \mathit{inc}\, 0$, one step per line, naming each. Then, what does *mult* compute in general? Why three parameters, where *add* had four?

$$\begin{aligned}
\mathit{mult}\, \bar{2}\, \bar{3}\, \mathit{inc}\, 0 &\rightsquigarrow \bar{2}\, (\bar{3}\, \mathit{inc})\, 0 && \mathit{mult}\\
&\rightsquigarrow \bar{3}\, \mathit{inc}\, (\bar{3}\, \mathit{inc}\, 0) && \bar{2}\\
&\rightsquigarrow \bar{3}\, \mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, 0))) && \bar{3}\\
&\rightsquigarrow \mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, (\mathit{inc}\, 0))))) && \bar{3}
\end{aligned}$$

Six unfoldings of *inc* gives 6. In general, f applied n times, m times over, is mn times. In the addition run, in line 3, there were three steps available since any of the three *inc*s could have been unfolded. The innermost was selected: another choice would **not** have given another answer. (See Theorem 1.8)

Theorem 1.8 (The Order of Steps)**.** Church-Rosser (1936): If a term runs to a normal form, it runs to only one.

Summary of Section 1.1

- A program is a finite list of equations and a term to evaluate
- To run it is to replace LHS by the RHS until none is left
- No store. No stack. No instruction pointer. No order of evaluation to settle.
- The conditional, the numbers, addition: built in front of us, out of nothing

1.2 Non-termination

Nothing in Definition 1.1 says what may be applied to what. A boolean is a function and a number is a function, so a function may be applied to either; and a function may be applied to itself. The definition

$$\omega x := xx$$

is legal: its RHS uses only its one parameter. So ω is the function that applies its argument to itself. Now, we apply that ω to itself, creating **the Monster**:

$$\Omega := \omega\omega,$$

and run it: There is one step available, the definition of ω with ω written for x, and it gives

$$\omega\omega \rightsquigarrow \omega\omega \rightsquigarrow \omega\omega \rightsquigarrow \dots$$

The step returns the term where it started. So there is a second step, and it returns it again, and no number of step ever finishes. The Monster is a nod to a function exhibited in 1872 by Weierstrass, continuous everywhere and differentiable nowhere: a perfectly legal construction that nobody who wrote the definitions had in mind.

Be exact about what that costs. Ω runs and there is no output, no error and no crash. The machine is working, and will still be working tomorrow. The failure here is not that the program returns the wrong answer - a wrong answer can be read and argued with. This one returned nothing and took the machine with it.

Nor is the loop a matter of notation. Compiled into constants, ω is **S I I**:

$$\begin{aligned} \mathbf{S}\,\mathbf{I}\,\mathbf{I}\,x &\rightsquigarrow \mathbf{I}\,x\,(\mathbf{I}\,x) && \mathbf{S} \text{ step} \\ &\rightsquigarrow^* x\,(\mathbf{I}\,x) && \mathbf{I}\,x \rightsquigarrow^* x \\ &\rightsquigarrow^* x\,x && \mathbf{I}\,x \rightsquigarrow^* x \end{aligned}$$

so Ω is **S I I**(**S I I**): several steps to come back to itself, instead of one. Compiling changes nothing. The loop is in the **shape**, not the notation.

Nor need a loop come back to where it started. Apply to itself a function that uses its argument three times,

$$\omega_3\,x := x\,x\,x, \qquad \Omega_3 := \omega_3\,\omega_3$$

and run it: $\omega_3\,\omega_3 \rightsquigarrow \omega_3\,\omega_3\,\omega_3 \rightsquigarrow \omega_3\,\omega_3\,\omega_3\,\omega_3 \rightsquigarrow \omega_3\,\omega_3\,\omega_3\,\omega_3\,\omega_3 \rightsquigarrow \cdots$ The term grows at every step and never repeats. So, a loop need not repeat itself, and a checker that watches for a repeat cannot catch them all.

Whether a run stops can even depend on the number of steps, which Theorem 1.8 seemed to make harmless. Consider

$$\mathbf{K}\,x\,\Omega,$$

in which two steps are available. The outer one, the **K** step, gives x back at once:

$$\mathbf{K}\,x\,\Omega \rightsquigarrow x$$

a normal form, and the run stops. The inner one, inside Ω, gives $\mathbf{K}\,x\,\Omega$ back, and taking it for ever never stops. There is no contradiction. The theorem says that a term has *at most one* normal form, and that no steps puts it out of reach — after the inner step, the outer one is still there to be taken — but that every order of steps reaches it.

Real languages differ in exactly this choice. A *lazy* language such as Haskell takes the outer step, and `const 1 undefined` is 1. Alternatively, a *strict* language such as Python

evaluates arguments first, so it takes the inner step: with `K = lambda a: lambda b: a` and `w = lambda x: x(x)`, the call `K(1)(w(w))` never returns `1`, and Python gives up only when the stack runs out.

Now look at the size of the thing. Ω is four symbols. It is not the end of a long chain of subtle mistakes but rather two sensible decisions in a row. Nothing in the three clauses of Definition 1.1 objected, and nothing in them could: they say strings are terms, and $\omega\omega$ is a perfectly valid string. The one rule of Definition 1.3 did not object either. It had a step available, and it took it.

So what can a checker achieve? A checker that catches *every* program that fails to stop **does not exist**, but we cannot yet prove this. What we can build is a checker that refuses a class of programs we can **describe exactly**, with Ω inside it.

1.3 Types and typings

What went wrong is not hard to name. The function ω takes an argument and returns a result, and nothing recorded *what kind* of result it gave back. Therefore, nothing was in a position to notice that when ω was handed ω, it had been handed something it was never built to take. So here is the main idea, and the rest of this chapter is its execution. Church's idea (1940): label every term with what it takes and returns; allow an application only when the argument's label is the one the function asks for; and check, before anything runs, that the labels fit together throughout.

Read a label $A \to B$ as a *contract*: give me an A, and I shall return a B. A contract can be checked without being honoured (i.e. without running anything), because it speaks of kinds and not of values, and a verdict in advanced is exactly what we want.

Three things must be settled before that is a checker rather than a hope:

1. What the labels are;
2. What it means for a term to carry one;
3. Which labellings are allowed.

The labels are called *types*, and they are built by two clauses, as a term was built by three.

Definition 1.9 (types)**.** Fix a stock of *basic types*, such as *Int*, *String*, and *Bool*; we write p, q, r, ... for basic types when it does not matter which. The *types* are defined inductively:

1. Every basic type p is a type.
2. If A and B are types, then $A \to B$ is a type: the *function type*, the type of functions that take an A and return a B.
3. Nothing else is a type.

In one line, we have $A, B ::= p \mid A \to B$. So $Int \to Bool$ is a test on integers, $(Int \to Int) \to Int$ is the type of something that takes a function, and $(p \to q) \to p$ is a type too.

We have written such types. In Java, `Function<A, B>` is $A \to B$. A method `static int square(int n)` has type $Int \to Int$, and the addition `m -> n -> m + n`, which takes its arguments one at a time, has type `Function<Integer, Function<Integer, Integer>>`, which is $Int \to Int \to Int$, nested to the right exactly as our arrow is.

For the arrow groups to the right: $A \to B \to C$ is $A \to (B \to C)$, a function of two arguments taking them one at a time, just as application grouped to the left. Every real checker has more — records, lists, generics — and every one of them has this.

We have the types; now we must say how to determine the type of a term, written $\vdash t : T$ and read "t has type T". But a term like $f(f\,x)$ has variables in it, and its type depends on theirs, so the claim has to carry a list of declarations of them, written Γ.

Definition 1.10 (context, judgement)**.** A *context* Γ is a finite list

$$x_1 : A_1, \ldots, x_n : A_n$$

pairing each variable in scope with a type, no variable listed twice; the empty list counts, and we write $\Gamma, x : A$ for Γ with the pair $x : A$ added at the end. The checker's basic claim is a *judgement*

$$\Gamma \vdash M : A,$$

read "M has type A when its variables have the types Γ gives them". When Γ is empty we simply write $\vdash M : A$, and then M is a program with type A outright.

Definition 1.11 (typing rules)**.** The same three clauses that built terms in Definition 1.1

$$\text{(K)} \quad \mathbf{K} : A \to B \to A \tag{1.1}$$
$$\text{(S)} \quad \mathbf{S} : (A \to B \to C) \to (A \to B) \to A \to C \tag{1.2}$$

where A, B and C stand for arbitrary types
In addition to that we have one look-up (variables)

$$\text{(var)} \quad x : T, \text{ if } x : T \text{ is declared in } \Gamma$$

and one match (applications)

$$\text{(app)} \quad M : A \to B \text{ and } N : A \text{ give } M\,N : B$$

Two schemas, one look-up, one match: That is the whole checker.

The types of **K** and **S**: Each schema is just what the combinator does, written as a contract. **K** keeps:

$$\mathbf{K}\,M\,N \rightsquigarrow M$$

Give it an A, then a B, it **keeps** the A and drops the B, so what comes out is an A: $\mathbf{K} : A \to B \to A$. Alternatively, **S** shares:

$$\mathbf{S}\,M\,N\,P \rightsquigarrow M\,P\,(N\,P)$$

It shares P with both. If $P : A$, then N must turn an A into B, and M must take an A and that B to a C: $\mathbf{S} : (A \to B \to C) \to (A \to B) \to A \to C$. Read the type off the rule: what goes in, what comes out.

Instances of a schema: (**K**) is one line, but gives **K** a type for every choice of A, B:

A	B	A type of **K**
Int	*Bool*	$Int \to Bool \to Int$
Bool	*Int*	$Bool \to Int \to Bool$
$Int \to Int$	*String*	$(Int \to Int) \to String \to (Int \to Int)$

Each one is of a typing **K**. A schema is what Java calls a generic

Definition 1.12 (a typing)**.** A typing of $\Gamma \vdash t : T$ is a finite list of judgements, all with the *same* context Γ,

$$\Gamma \vdash t_1 : T_1, \ldots, \Gamma \vdash t_n : T_n$$

ending in $\Gamma \vdash t : T$, each line justified by (**K**), (**S**), (var) or (app) from the earlier lines. Nothing else is allowed. To write justifications, "(app) i, j": line i is the function, line j the argument. After (**K**) or (**S**): in brackets, the types put for A, B, C.

Example 1.13 (the empty context)**.** Recall: `if b then M else N` is just $b\,M\,N$, and $\mathit{true} := \mathbf{K}$. So type $\mathbf{K}\,3\,4$. No variables, so nothing to declare: Γ is empty.

LINE	JUDGEMENT	JUSTIFICATION
1.	$\vdash \mathbf{K} : \mathit{Int} \to \mathit{Int} \to \mathit{Int}$	(K) [*Int*, *Int*]
2.	$\vdash 3 : \mathit{Int}$	ready-made
3.	$\vdash \mathbf{K}\,3 : \mathit{Int} \to \mathit{Int}$	(app) 1, 2
4.	$\vdash 4 : \mathit{Int}$	ready-made
5.	$\vdash \mathbf{K}\,3\,4 : \mathit{Int}$	(app) 3, 4

An Int, and we know it before anything runs. If we swap 4 for `"hello"`: choose $B := \mathit{String}$ in line 1, and $\mathbf{K}\,3$ `"hello"`:*Int* too. **K** does not care what it drops.

Example 1.14 (a non-empty context)**.** $\mathit{twice}\,f\,x := f(f\,x)$. To type its body, declare its parameters: $\Gamma = f : \mathit{Int} \to \mathit{Int}$, $x : \mathit{Int}$

LINE	JUDGEMENT	JUSTIFICATION
1.	$\Gamma \vdash f : \mathit{Int} \to \mathit{Int}$	(var)
2.	$\Gamma \vdash x : \mathit{Int}$	(var)
3.	$\Gamma \vdash f\,x : \mathit{Int}$	(app) 1, 2
4.	$\Gamma \vdash f\,(f\,x) : \mathit{Int}$	(app) 1, 3

Γ is the parameter list. Line 1 is cited twice, which is allowed. There is no case for the name twice itself, Chapter 2 deals with that.

Example 1.15 (the identity, typed)**.** Type $\mathbf{I} = \mathbf{S}\ \mathbf{K}\ \mathbf{K}$ at $A \to A$. Write P for $A \to A$

LINE	JUDGEMENT	JUSTIFICATION
1.	$\vdash \mathbf{S} : (A \to P \to A) \to (A \to P) \to P$	(S) [A, P, A]
2.	$\vdash \mathbf{K} : A \to P \to A$	(K) [A, P]
3.	$\vdash \mathbf{S}\,\mathbf{K} : (A \to P) \to P$	(app) 1, 2
4.	$\vdash \mathbf{K} : A \to P$	(K) [A, A]
5.	$\vdash \mathbf{S}\,\mathbf{K}\,\mathbf{K} : A \to A$	(app) 3, 4

Line 1 is the one nobody writes unprompted: it is found backwards from the goal. The search takes a sentence; the checks take none.

Work backwards from the goal. Leave the letters of (S) unknown, and the two **K**s decide them.

1. Goal: $\mathbf{S}\,\mathbf{K}\,\mathbf{K} : A \to A$
2. (S) gives $\mathbf{S} : (X \to Y \to Z) \to (X \to Y) \to X \to Z$, so $\mathbf{S}\,\mathbf{K}\,\mathbf{K} : X \to Z$. Hence $X = A$ and $Z = A$.
3. The first **K** must have type $A \to Y \to A$. That has the shape $V \to W \to V$ forces $Y = W \to A$, for some W.

4. The second **K** must have type $A \to Y$. The shape $V \to W \to V$ forces $Y = W \to A$, for some W
5. Any W will do. Choose $W = A$: then $Y = A \to A$, which is P. That is line 1.

Example 1.16 (The red line)**.** The line we walked in to lecture: `total = 3 + "hello"`, with `"hello"` underlined in red. expected Int, got String. Without types, nothing stops it: inside the machine 3 and `"hello"` are both just bits, and the processor would add them.

LINE	JUDGEMENT	JUSTIFICATION
1.	$\vdash +$: Int $\to$ Int $\to$ Int	ready-made
2.	$\vdash 3$: Int	ready-made
3.	$\vdash +3$: Int $\to$ Int	(app) 1, 2
4.	$\vdash$ `"hello"` : String	ready-made
5.	(app) 3, 4? wants *Int*, gets *String*: **no match**	

No rule applies, so no typing exists; refused, before anything runs. That is the whole content of the editor, minus the decorations.

Java says the same thing in its own words. Java's own + would just turn `3` and `"hello"` into the string `"3hello"`, so use an addition typed like ours, Int $\to$ Int $\to$ Int
`Function<Integer, Function<Integer, Integer>> plus = m -> n -> m + n;`
`int total = plus.apply(3).apply("hello")` and the compiler gives an error.

Theorem 1.17 (subject reduction)**.** If $\Gamma \vdash M : A$ and $M \rightsquigarrow N$ by a **K** step, an **S** step or a ready-made step, then $\Gamma \vdash N : A$

The **K** case: Suppose $\mathbf{K}\,P\,Q : A$ and $\mathbf{K}\,P\,Q \rightsquigarrow P$. Why is $P : A$? Read the typing backward twice.

1. $\mathbf{K}\,P\,Q : A$ came by (app) from $\mathbf{K}\,P : B \to A$ and $Q : B$.
2. $\mathbf{K}\,P : B \to A$ came by (app) from $\mathbf{K} : C \to B \to A$ and $P : C$.
3. (K) gives **K** only types $V \to W \to V$, so $C = A$.

Lemma 1.18 (the types of **S K K**)**.** The goal: show that the Monster has **no** type. Then the checker refuses it, and the loop never runs. There are infinitely many types, we cannot try them one by one. Instead, we read a typing **backwards**. Each kind of term has exactly one rule that could have produced it, so any typing of a term is forced to have a certain shape.

$$\text{If } \Gamma \vdash \mathbf{S}\ \mathbf{K}\ \mathbf{K} : T \text{ then } T \text{ is } U \to U \text{ for some type } U$$

The last line came by (app), so four earlier lines exist:

$$\mathbf{S} : A'' \to A' \to T, \quad \mathbf{K} : A'', \mathbf{K} : A', \text{ and } \mathbf{S}\ \mathbf{K} : A' \to T$$

1. (S): $A'' = X \to Y \to Z, A' = X \to Y, T = X \to Z$
2. (K) has shape $V \to W \to V$; A'' has it, so $Z = X$
3. $A' = X \to Y$ has it too, so $Y = W \to X$.

Hence, $T = X \to Z = X \to X$. The backwards reading is the shape of every proof in Chapter 2.

Proposition 1.19 (the Monster has no type)**.** Recall that $\omega = \mathbf{S\ I\ I}$ and $\Omega = \omega\omega = \mathbf{S\ I\ I}(\mathbf{S\ I\ I})$. To show that the Monster has no type, it is enough to show that $\omega = \mathbf{S\ I\ I}$ has none: (app) needs typing of both M and N. No type for M, no type for $M\,N$. There is no context Γ and no type T with $\Gamma \vdash \mathbf{S\ I\ I} : T$. Consequently, Ω has no typing.

Proof of Proposition 1.19: Suppose $\Gamma \vdash \mathbf{S\ I\ I}\ : T$, and read the typing backwards.

1. The last line is (app), so there are typings of $\mathbf{S\ I}$ and of $\mathbf{I}$; and $\mathbf{S\ I}$ comes by (app) from $\mathbf{S}$ and $\mathbf{I}$
2. (S) then wants the first $\mathbf{I}$: $X \to Y \to Z$ and the second $\mathbf{I}$: $X \to Y$.
3. Lemma 1.18: each has the shape $U \to U$. So $X = Y \to Z$ and $X = Y$.

$$Y = Y \to Z$$

Remember this equation. $Y \to Z$ is strictly longer than Y, and equal types have equal length. No type is a proper part of itself. The parameter was the problem, and compiling it away did not remove it.

Proposition 1.16 refuses one looping program: the Monster. But there are infinitely many programs. Could a cleverer one loop forever and still get a type? The claim: **no typed program loops**. The question: do typed programs always run to a **normal form**? (Definition 1.3)

Theorem 1.20 (normalisation)**.** Every program with a typing runs to a normal form. A typed program cannot loop: if the checker says yes, the run finishes. With Theorem 1.17, a yes means no step goes wrong and the run ends.

1.4 Summary of Chapter 1

- Under every language we use is a combinatory algebra: two constants and application, run by two rewrite rules. Unpoliced, it computes everything, including a program that never stops.
- A type is a contract, a basic type or $A \to B$. The checker allows an application only when the contract is met.
- Its certificate is a typing: a finite list of judgements $\Gamma \vdash M : A$, each justified by (**K**), (**S**), (var), or (app).
- Its *no* is (app) failing to match. Its *yes* names no input, so covers them all, and by subject reduction still holds as the program runs.